

501. Find Mode in Binary Search Tree

PLATFORM

Platform: LeetCode

DIFFICULTY

EASY

DATE

Solved: June 10, 2026

Problem Summary

Given the root of a BST, return all the **mode(s)** (the value(s) that appear most frequently).

A BST's inorder traversal produces values in **sorted order**, which is the key observation.

Approach

Step1 : Perform Inorder Traversal

Step2 : Count Consecutive Occurrences

Step3 : Update modes

Step4 : Process Last Group

Step5 : Convert List to Array

Algorithm

1. Perform inorder traversal.
2. Store values in a list.
3. Scan the sorted list.
4. Count consecutive equal values.
5. Maintain: count, maxCount, modes
6. Process the last group.
7. Convert modes list to array.
8. Return answer.

Time Complexity & Space Complexity

TC : $O(n)$

SC : $O(n)$

The screenshot displays the LeetCode problem page for "501. Find Mode in Binary Search Tree". The problem description states: "Given the root of a binary search tree (BST) with duplicates, return all the mode(s) (i.e., the most frequently occurred element) in it. If the tree has more than one mode, return them in any order. Assume a BST is defined as follows: The left subtree of a node contains only nodes with keys less than or equal to the node's key. The right subtree of a node contains only nodes with keys greater than or equal to the node's key. Both the left and right subtrees must also be binary search trees." Example 1 shows a tree with root 1 and left child 2, with input [1, null, 2, 2] and output [2]. Example 2 shows a tree with root 1 and no children, with input [1] and output [1]. Constraints include: "The number of nodes in the tree is in the range [1, 10^4]" and "-10^5 <= Node.val <= 10^5". The solution is provided in Java, showing an inorder traversal that counts consecutive values and updates the modes list. The code is as follows:

```
1 // Java
2 class Solution {
3     public int[] findMode(TreeNode root) {
4         List<Integer> list = new ArrayList<>();
5         List<Integer> modes = new ArrayList<>();
6         inorder(root, list);
7         if (list.size() == 0) return new int[0];
8         int count = 1;
9         int maxCount = 1;
10         for (int i = 1; i < list.size(); i++) {
11             if (list.get(i).equals(list.get(i - 1))) {
12                 count++;
13             } else {
14                 if (count > maxCount) {
15                     maxCount = count;
16                     modes.clear();
17                     modes.add(list.get(i - 1));
18                 } else if (count == maxCount) {
19                     modes.add(list.get(i - 1));
20                 }
21                 count = 1;
22             }
23         }
24         // Process the last group
25         if (count > maxCount) {
26             maxCount = count;
27             modes.clear();
28             modes.add(list.get(list.size() - 1));
29         } else if (count == maxCount) {
30             modes.add(list.get(list.size() - 1));
31         }
32         // Handle case where all values are unique
33         if (modes.isEmpty()) {
34             modes.add(list.get(0));
35         }
36         for (int i = 1; i < list.size(); i++) {
37             if (!list.get(i).equals(list.get(i - 1))) {
38                 modes.add(list.get(i));
39             }
40         }
41         int[] ans = new int[modes.size()];
42         for (int i = 0; i < modes.size(); i++) {
43             ans[i] = modes.get(i);
44         }
45         return ans;
46     }
47     private void inorder(TreeNode root, List<Integer> list) {
48         if (root == null) return;
49         inorder(root.left, list);
50         list.add(root.val);
51         inorder(root.right, list);
52     }
53 }
```